

RAG 技术详解与实践应用

第3讲：大模型怎么玩？用LazyLLM带你理解调用逻辑与Prompt魔法



目录

-  1. 上节回顾
-  2. LazyLLM介绍
-  3. 在LazyLLM中使用大模型
-  4. LazyLLM中的数据流
-  5. 借助数据流搭建你的RAG应用



RAG环境准备

- 安装Python & pip
- 安装LazyLLM
- 获取API key
- 检查sqlite是否支持多线程

RAG基本组件简介

- 文档管理
- 检索模块
- 生成模块

第一个RAG程序

- RAG代码详解
- 展望：更准，更快，更强



目录



1. 上节回顾



2. LazyLLM介绍



3. 在LazyLLM中使用大模型



4. LazyLLM中的数据流



5. 借助数据流搭建你的RAG应用



LazyLLM简介



析助手

解读复杂文件，提
分析，帮助理清投



法律文件分析

快速分析法律文件，提取关键信息，辅助律师进行案件研究和法律文书撰写



教学内容生成

根据教学大纲和学生学习进度，自动生成教学内容和练习题，辅助教师进行个性化教学



医疗诊断助手

利用大模型分析医疗影像数据，辅助医生进行疾病诊断，提供个性化治疗方案



实时更新金
炼重点内容
资逻辑。

一键部署

langchain + llamaindex = ?



灵活微调

轻松上手



LazyLLM

一站式多Agent应用开发工具

拒绝过度封装



铁路知识问答

通过图片视频，理解铁路状况，提供铁路对应知识，帮助确认问题原因



财务分析助手

实时更新财务数据库，辅助解读复杂文件，提炼重点内容，提供财务数据分析，帮助理清投资逻辑



旅游规划顾问

结合用户偏好和旅行历史，提供个性化旅游规划建议，包括景点推荐、行程安排等



物流优化专家

通过分析物流数据，优化配送路线和库容，降低物流成本，提高配送效率

LazyLLM开发的初衷

原型
搭建

数据
分析

迭代
优化

LazyLLM

一款构建多Agent大模型应用的开发框架，协助开发者用极低的成本构建复杂的AI应用，并可以持续的迭代优化效果。

AI Native 产品开发范式

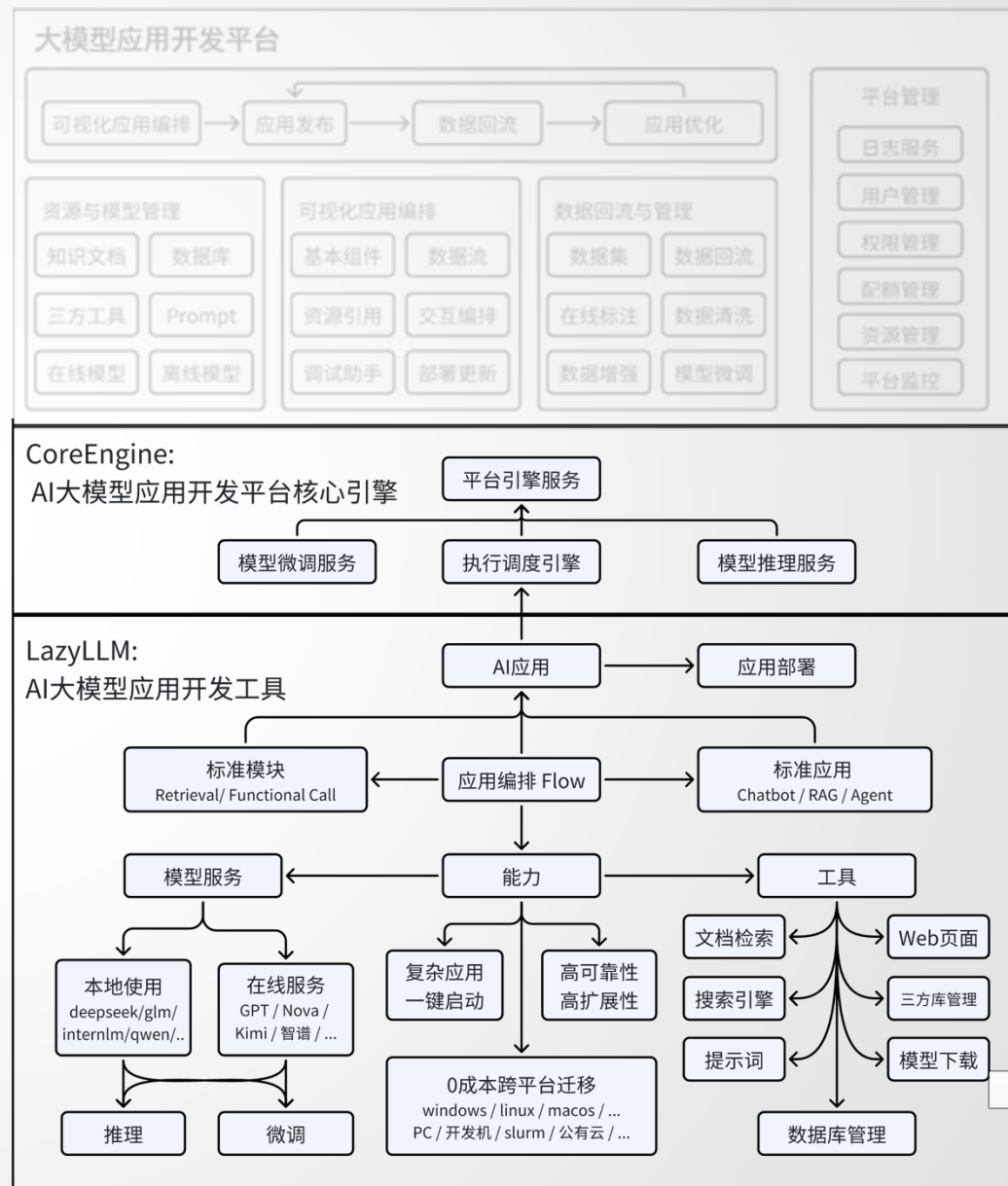
基于模型开发实验科学的开发范式设计，先基于LazyLLM快速跑通应用的原型，结合场景任务数据进行bad-case分析，然后对应用中的关键环节进行算法迭代和模型微调，逐步提升应用效果。

产品体验及使用

适用大模型产品开发者、大模型爱好者以及热爱编程的同学。

目前，LazyLLM 已经在GitHub上开源，**欢迎大家打星**

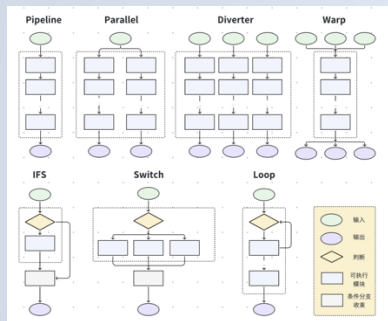
- 项目地址: <https://github.com/LazyAGI/LazyLLM>
- 项目文档: docs.lazyllm.ai



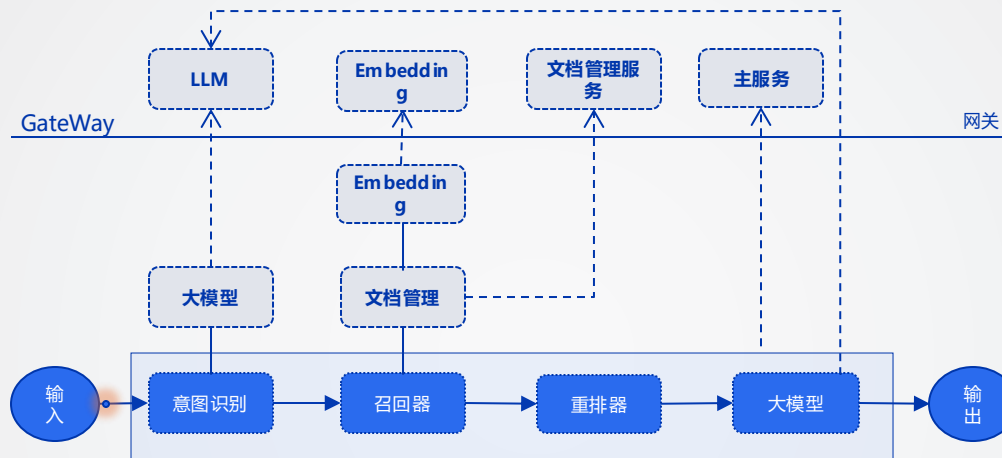
核心亮点

以数据流为核心的应用开发范式

LazyLLM支持Pipeline、Parallel、Switch、If、Loop、Diverter、Warp、Graph等数据流



复杂应用一键部署



跨平台

利用轻量网关实现分布式应用一键部署，助力用户快速实现产品落地

兼容多个操作系统



兼容多种IaaS平台

- 裸金属、K8S、Slurm、公有云

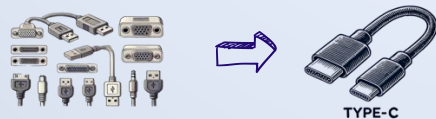
简洁代码设计

精细化模块设计，符合直觉的代码风格帮你集中精力打造爆款产品



不同技术选型间的统一体验

对齐同一模块在不同技术选型上的使用体验，让您灵活试验各种基模型、训推框架与数据库



RAG场景的灵活适配

- 轻松可调的Retriever与Reranker组件
- 多embedding检索与自定义相似度计算方式
- 自定义node_group，从不同维度对文档进行检索和打分

多Agent编排

- 内置封装FunctionCall、React、ReWOO、PlanAndSolve等多种Agent
- 灵活调配不同组件进行功能拓展
- 简单的工具配置与使用



核心亮点：简洁的代码设计

$$\text{Softmax}(x_i) = \frac{\exp(x_i)}{\sum_j \exp(x_j)}$$

$$\text{LogSoftmax}(x_i) = \log \left(\frac{\exp(x_i)}{\sum_j \exp(x_j)} \right)$$



```
1 import math
2 from lazyllm import pipeline, warp, bind, _0, ifs, Identity
3
4 def softmax(log: bool = False):
5     with pipeline() as p:
6         p.exp = warp(math.exp).aslist
7         p.sum = sum
8         p.div = warp(lambda x, y: x / y, ifs(
9             log, math.log, Identity)) | bind(p.exp, y=_0)
10    return p
```



```
1 import math
2 from langchain_core.runnables import chain, RunnablePassthrough, RunnableLambda
3 from langchain_core.runnables.base import RunnableEach
4
5 def softmax(log: bool = False):
6     softmax_chain = (
7         RunnableEach(bound=RunnableLambda(math.exp)) |
8         {"sum": RunnableLambda(sum), "exp": RunnablePassthrough()} |
9         RunnableLambda(lambda inputs:
10             RunnableEach(bound=RunnableLambda(lambda x, y: x / y)).bind(
11                 y=inputs.get("sum")).invoke(inputs.get("exp"))) |
12         RunnableEach(bound=RunnableLambda(
13             lambda z, log: math.log(z) if log else z)).bind(log=log)
14     )
15    return softmax_chain
```



```
1 import math
2 from typing import Any, List
3 from llama_index.core.workflow import Context, Event, StartEvent, StopEvent, Workflow, step
4
5 class ExpEvent(Event):
6     exp: List[float]
7
8 class SumEvent(Event):
9     sum: float
10
11 class SoftmaxEvent(Event):
12     result: List[float]
13
14 class SoftmaxFlow(Workflow):
15     def __init__(self, log: bool = False, *args: Any, **kwargs: Any) -> None:
16         super().__init__(*args, **kwargs)
17         self.log = log
18
19     @step
20     async def exponentiation(self, ev: StartEvent) -> ExpEvent:
21         inputs = ev.inputs
22         return ExpEvent(exp=[math.exp(x) for x in inputs])
23
24     @step
25     async def summation(self, ev: ExpEvent) -> SumEvent:
26         exp = ev.exp
27         return SumEvent(sum=sum(exp))
28
29     @step
30     async def division(self, ctx: Context, ev: ExpEvent | SumEvent) -> SoftmaxEvent | None:
31         data = ctx.collect_events(ev, [ExpEvent, SumEvent])
32         if not data:
33             return None
34         return SoftmaxEvent(result=[x / data[1].sum for x in data[0].exp])
35
36     @step
37     async def log_if_need(self, ev: SoftmaxEvent) -> StopEvent:
38         return StopEvent(result=[math.log(x) for x in ev.result] if self.log else ev.result)
```

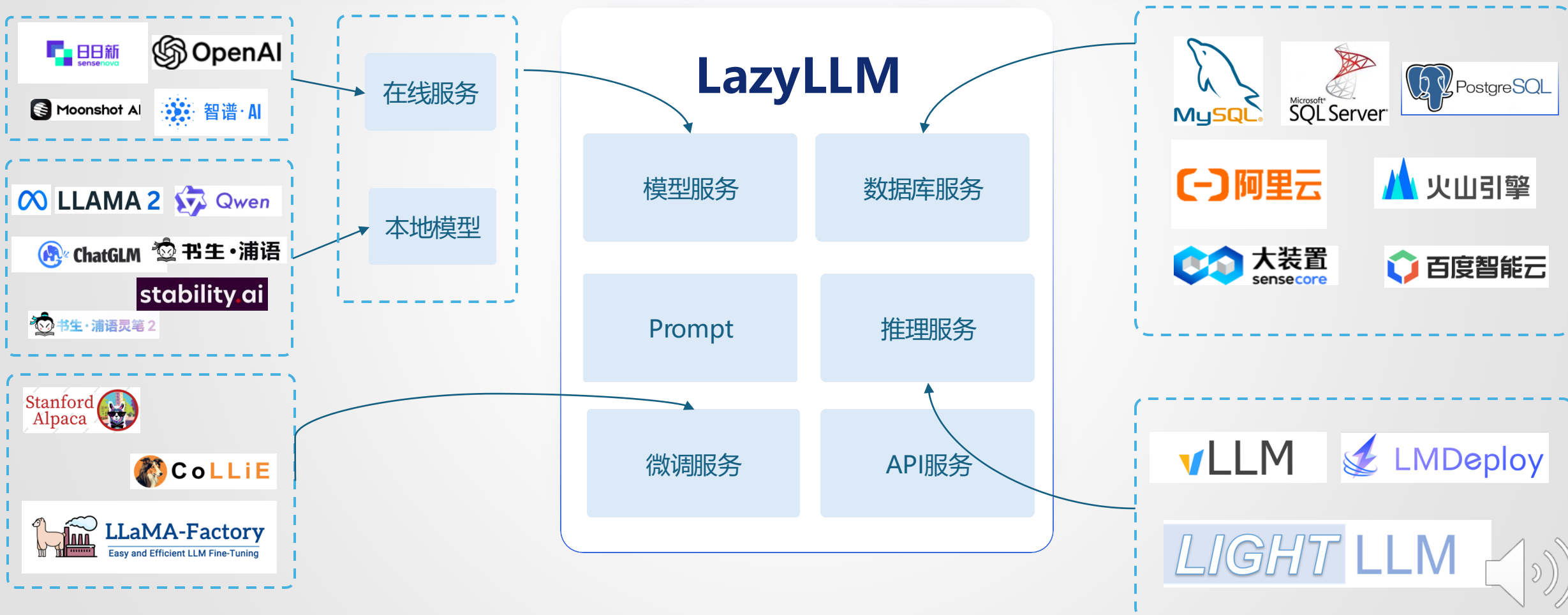


核心亮点：不同技术选型间的统一体验

统一调用

统一服务

统一部署



了解更多



扫码加入微信交流群



扫码关注公众号



目录

-  1. 上节回顾
-  2. LazyLLM介绍
-  3. 在LazyLLM中使用大模型
-  4. LazyLLM中的数据流
-  5. 借助数据流搭建你的RAG应用



从0开始搭建环境，包括安装Python和Pip，以及配置环境隔离，可以参考第二讲 10分钟上手一个最小可用RAG系统

从pip安装

1. `pip install lazyllm`

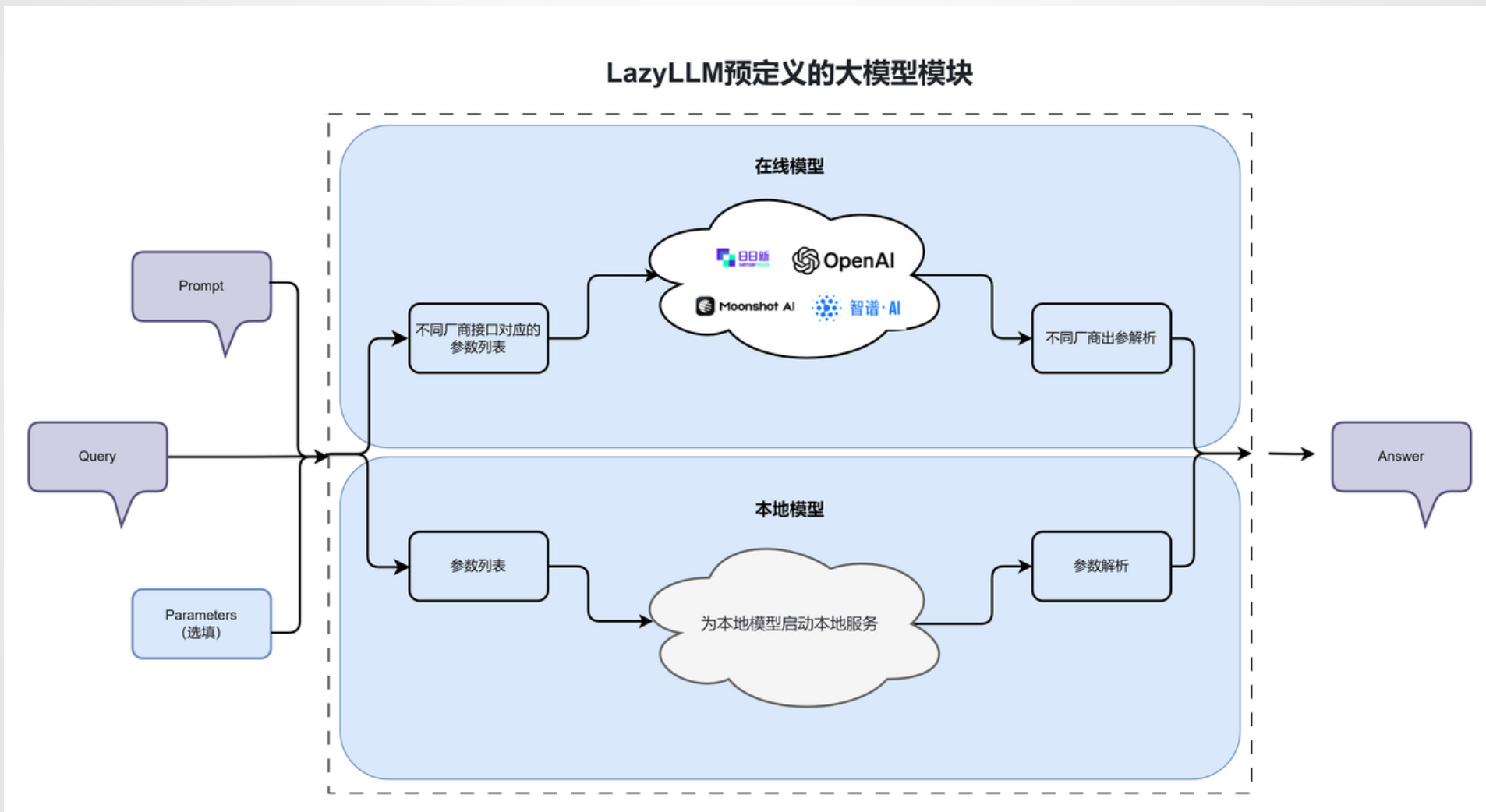
从源码安装

1. `git clone https://github.com/LazyAGI/LazyLLM.git`
2. `cd LazyLLM`
3. `pip3 install -r requirements.txt`
4. `export PYTHONPATH=$PWD:$PYTHONPATH`

```
now.py / ...
1  from lazyllm import parallel
2  |
3  def f1(input): return input
4
5  with parallel() as p:
6      p (type) parallel = Parallel
7      p
8  asser A class for managing parallel flows in LazyLLMFlows.
9
10 This class inherits from LazyLLMFlowsBase and provides an interface for running or
11 execution using threads and allows for the return of results as a dictionary.
12
13 The Parallel class can be visualized as follows:
14
15 text
16 #      /> module11 -> ... -> module1N -> out1 \
17 # input -> module21 -> ... -> module2N -> out2 -> (out1, out2
18 #      \> module31 -> ... -> module3N -> out3 /
19
20 The Parallel.sequential method can be visualized as follows:
21
22 text
```



在LazyLLM中使用大模型



在线模型的使用



在线模型的使用

The screenshot displays a web-based IDE interface. On the left, the 'EXPLORER' sidebar shows a file tree with 'EXAMPLES' expanded, containing 'data_kb', 'data_prepare.py', 'llm.py' (selected), and 'rag.py'. The main editor area shows the 'llm.py' file with a single line of code: '1'. Below the editor, the 'TERMINAL' tab is active, showing a shell prompt: 'o (lazyllm) macro@CNBJWZHM examples %'. The bottom status bar indicates the current line and column (Ln 1, Col 1), file encoding (UTF-8), line feed (LF), and the active Python environment (3.10.0 ('lazyllm')).



在线大模型和本地大模型的基础使用

先通过环境变量配置好API key

在线大模型

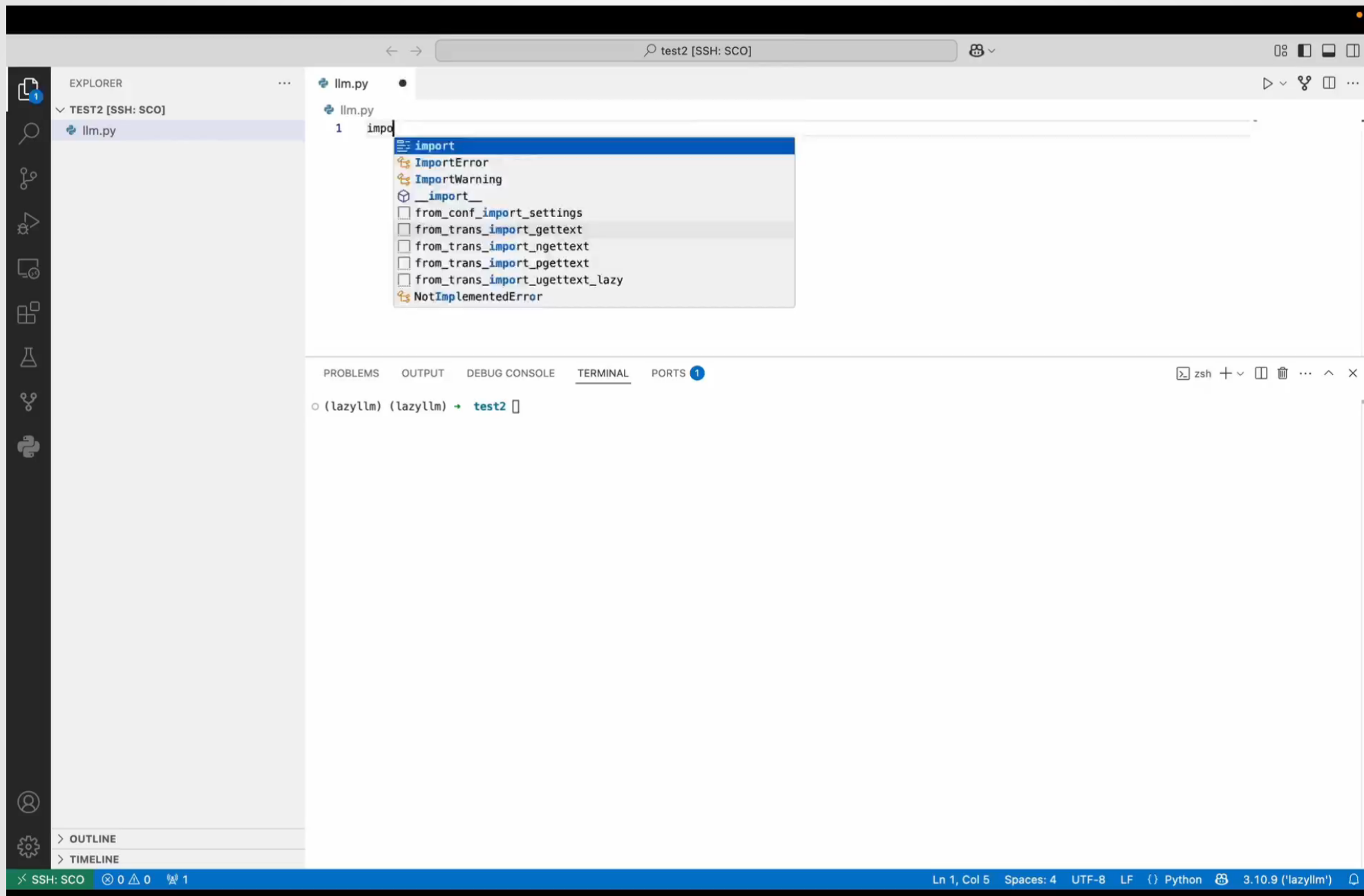
1. `import lazyllm`
2. `llm = lazyllm.OnlineChatModule(source='sensenova')`
3. `print(llm('你是谁'))`

本地大模型

1. `import lazyllm`
2. `llm = lazyllm.TrainableModule('internlm2-chat-7b')`
3. `llm.start()`
4. `print(llm('你是谁' ))`



本地模型的使用



流式输出

The screenshot displays the Visual Studio Code (VS Code) interface with a remote connection established to a machine named 'lazyllm'. The interface is divided into several sections:

- REMOTE EXPLORER (Left Panel):** Shows the 'Remotes (Tunnels/SSH)' section. Under the 'SSH' category, several remote machines are listed, including '1424', 'DEV', 'SCO', 'test2 test2', 'test test', 'lazyllm lazyllm', 'wangzhihong wangzhihong', 'CI', and 'JumpMachine'. The 'lazyllm lazyllm' machine is selected.
- Editor (Main Area):** Displays a file named 'llm.py' with a single line of code: `1`. The file is open in a tab titled 'llm.py'.
- Terminal (Bottom Panel):** Shows the terminal output for the 'lazyllm' machine. The prompt is `(lazyllm) macro@CNBJWZHM examples %`, indicating the user is in the 'examples' directory.
- Bottom Status Bar:** Displays the current file's status: 'Ln 1, Col 1', 'Spaces: 4', 'UTF-8', 'LF', 'Python', and '3.10.0 ('lazyllm')'.



本地大模型复用

用法1： 在同一个进程内，通过**share**来实现大模型的共享

```
m = lazyllm.TrainableModule('internlm2-chat-7b', stream=True).start()
r = stream_call(m, '你好,请帮我写一篇300字的作文,以知识库为题目')
m2 = m.share(prompt= '请帮我对输入进行总结: ')
```

用法2： 在不同进程中，通过制定**推理框架**和**url**来实现大模型的共享

```
import lazyllm
m = lazyllm.TrainableModule('internlm2-chat-7b').deploy_method(
    lazyllm.deploy.lightllm, url='http://10.119.17.169:36846/generate')
```



本地大模型复用

The screenshot shows a VS Code editor window titled 'test2 [SSH: SCO]'. The Explorer sidebar on the left shows a file named 'llm.py' under a folder 'TEST2 [SSH: SCO]'. The main editor area displays the following Python code:

```
1 llm.py > stream_call
2 from functools import partial
3
4 def stream_call(m, query):
5     with lazyllm.ThreadPoolExecutor(1) as executor:
6         lazyllm.FileSystemQueue().dequeue()
7         future = executor.submit(partial(m, llm_chat_history=[]), query)
8         while True:
9             if value := lazyllm.FileSystemQueue().dequeue():
10                 print(f'output: {"".join(value)}')
11             elif future.done():
12                 break
13         print(f'result: {future.result()}')
14
15 m = lazyllm.TrainableModule('internlm2-chat-7b', stream=True).start()
16 stream_call(m, '你好,请帮我写一篇300字的作文,以知识库为题目')
```

Below the code editor, the TERMINAL panel is active, showing a shell prompt: `(lazyllm) (lazyllm) → test2`. The status bar at the bottom indicates the file is at line 13, column 37, with 11 characters selected. It also shows the file encoding as UTF-8, line endings as LF, and the Python version as 3.10.9.



本地大模型复用

用法1：在同一个进程内，通过share来实现大模型的共享

1. `m = lazyllm.TrainableModule('internlm2-chat-7b', stream=True).start()`
2. `r = stream_call(m, '你好,请帮我写一篇300字的作文,以知识库为题目')`
3. `m2 = m.share(prompt= '请帮我对输入进行总结: ')`

用法2：在不同进程中，通过制定推理框架和url来实现大模型的共享

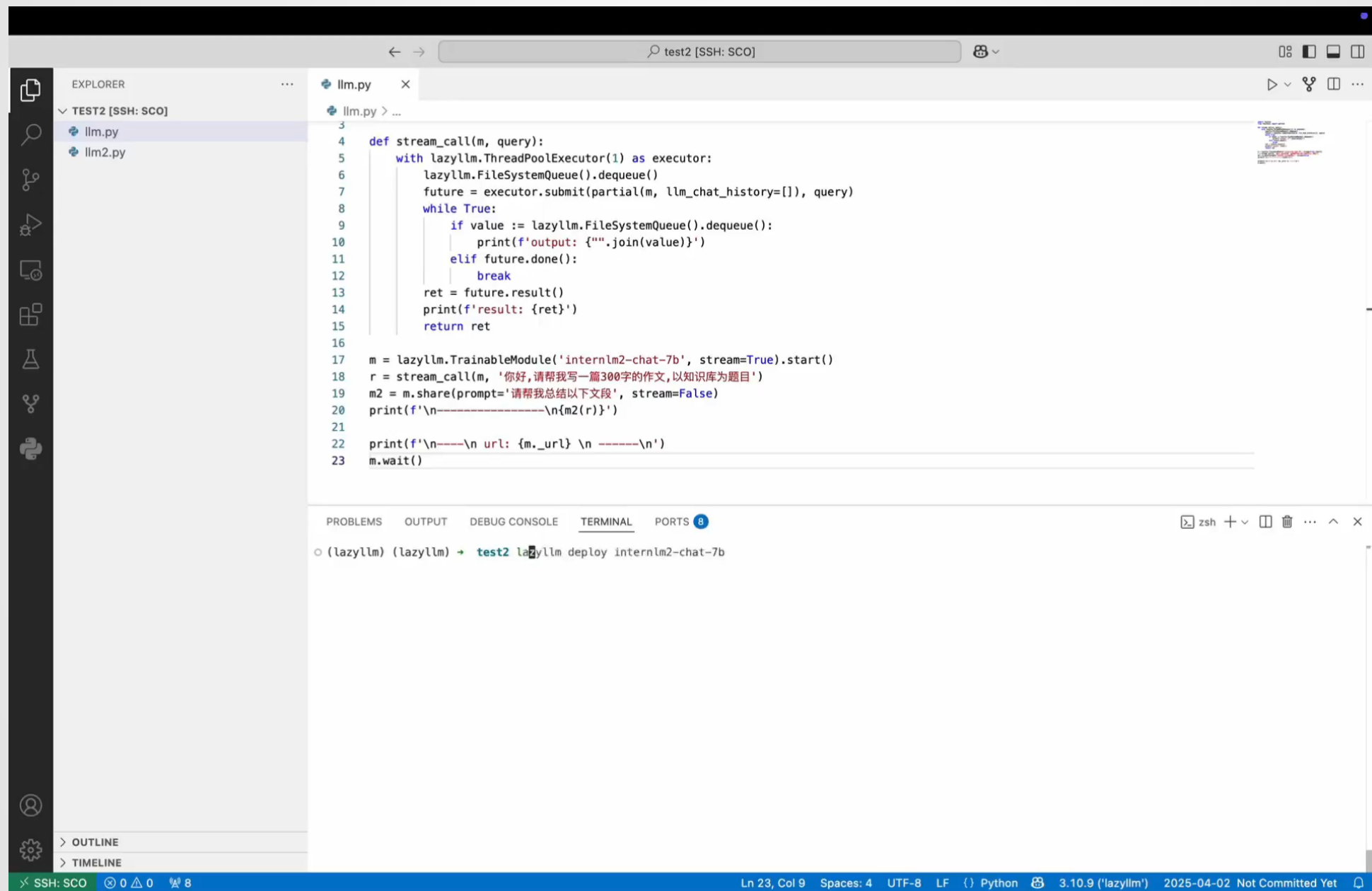
1. `import lazyllm`
2. `m = lazyllm.TrainableModule('internlm2-chat-7b').deploy_method(`
3. `lazyllm.deploy.lightllm, url='http://10.119.17.169:36846/generate')`

小彩蛋：除了用法1在代码中起推理服务之外，我们还提供了命令行工具

1. `$ lazyllm deploy internlm2-chat-7b`



本地大模型复用



```
3
4 def stream_call(m, query):
5     with lazyllm.ThreadPoolExecutor(1) as executor:
6         lazyllm.FileSystemQueue().dequeue()
7         future = executor.submit(partial(m, llm_chat_history=[]), query)
8         while True:
9             if value := lazyllm.FileSystemQueue().dequeue():
10                 print(f'output: {"".join(value)}')
11                 elif future.done():
12                     break
13             ret = future.result()
14             print(f'result: {ret}')
15             return ret
16
17 m = lazyllm.TrainableModule('internlm2-chat-7b', stream=True).start()
18 r = stream_call(m, '你好,请帮我写一篇300字的作文,以知识库为题目')
19 m2 = m.share(prompt='请帮我总结以下文段', stream=False)
20 print(f'\n-----\n{m2(r)}')
21
22 print(f'\n----\n url: {m._url} \n ----\n')
23 m.wait()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS 8

o (lazyllm) (lazyllm) → test2 lazyllm deploy internlm2-chat-7b

SSH: SCO 0 0 8 Ln 23, Col 9 Spaces: 4 UTF-8 LF () Python 3.10.9 ('lazyllm') 2025-04-02 Not Committed Yet



Prompt - Alpaca格式

```
{  
  "instruction": "基于以下背景知识进行回答。",  
  "input": "背景知识：机器学习模型的复杂度越高，越容易过拟合。",  
  "output": "是的，模型复杂度过高容易导致过拟合，因此需要正则化等手段来限制模型复杂度。"  
}
```



```
"""  
Below is an instruction that describes a task, paired with extra messages such as input that provides further  
context if possible. Write a response that appropriately completes the request.\n\n### Instruction: 基于以下背景知识进行回答.\n\n### Input: 背景知识：机器学习模型的复杂度越高，越容易过拟合.\n\n### Output: 是的，模型复杂度过高容易导致过拟合，因此需要正则化等手段来限制模型复杂度。  
"""
```



Prompt - Chat格式

```
[  
  {"role": "system", "content": "你是一个博学多闻的助手。"},  
  {"role": "user", "content": "什么是机器学习中的过拟合?"},  
  {"role": "assistant", "content": "过拟合是指模型在训练数据上表现很好, 但在新数据上表现不佳。"}  
]
```



```
""\  
<|im_start|>system\nYou are an AI assistant in the language chosen by the user such as English and 中文. 你是一个博学多闻的助手。 <|im_end|>\n\n<|im_start|>user\n什么是机器学习中的过拟合?  
<|im_end|>\n<|im_start|>assistant\n过拟合是指模型在训练数据上表现很好, 但在新数据上表现不佳。 <|im_end|>\n""
```



Prompt - 格式对比

	Alpaca格式	Chat格式
适合场景	单轮问答、指令微调	多轮对话、复杂任务
上下文处理	单轮任务、无上下文记忆	支持上下文记忆、连续对话
结构复杂度	简单	灵活多变
对话角色	单角色	多角色 (system, user, assistant)



Prompt - Chat格式

```
[  
  {"role": "system", "content": "你是一个博学多闻的助手。"},  
  {"role": "user", "content": "什么是机器学习中的过拟合? "},  
  {"role": "assistant", "content": "过拟合是指模型在训练数据上表现很好, 但在新数据上表现不佳。"}  
]
```

在线格式



本地格式

```
""\n<|im_start|>system\nYou are an AI assistant in the language chosen by the user such as English and 中文. 你是一个博学多闻的助手。 <|im_end|>\n\n\n<|im_start|>user\n什么是机器学习中的过拟合?\n<|im_end|>\n\n<|im_start|>assistant\n过拟合是指模型在训练数据上表现很好, 但在新数据上表现不佳。 <|im_end|>\n""
```



Prompt - 格式对比

	Alpaca格式	Chat格式
适合场景	单轮问答、指令微调	多轮对话、复杂任务
上下文处理	单轮任务、无上下文记忆	支持上下文记忆、连续对话
结构复杂度	简单	灵活多变
对话角色	单角色	多角色 (system, user, assistant)

	在线格式	本地格式
适合场景	调研在线模型	本地启动推理服务
格式	Json	字符串
特点	会区分角色	会包含特殊标记，如< im_start >

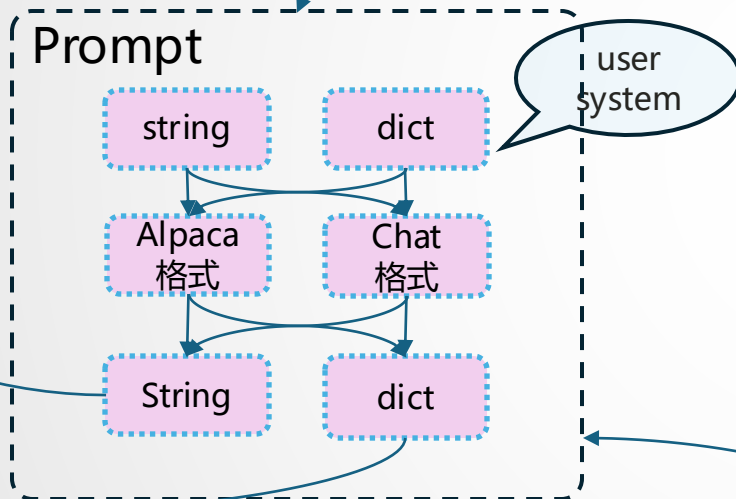


在大模型中使用Prompt

```
from lazyllm import TrainableModule, OnlineChatModule
```

```
m = TrainableModule('internlm2-chat-7b', stream=True).prompt(...).formatter(...)
```

```
m(query)
```



本地模型和在线模型使用Prompt的方式 **完全一致!**

```
m = OnlineChatModule('openai', model='gpt-3.5-turbo', stream=True).prompt(...).formatter(...)
```

```
m(query)
```



在大模型中使用Prompt

```
class UrlModule(ModuleBase, UrlTemplate):
    293
    351 # Cannot modify or add any attribute of self
    352 # prompt keys (excluding history) are in __input (ATTENTION: dict, not kwargs)
    353 # deploy parameters keys are in **kw
    354 def forward(self, __input: Union[Tuple[Union[str, Dict], str], str, Dict] = package(), # noqa C901
    355             *, llm_chat_history=None, lazyllm_files=None, tools=None, stream_output=False, **kw):
    356     assert self._url is not None, f'Please start {self.__class__} first'
    357     stream_output = stream_output or self._stream
    358     url = self._url
    359
    360
    361     if self.template_message:
    362         if isinstance(__input, package()):
    363             assert not lazyllm_files, 'Duplicate `files` argument provided by args and kwargs'
    364             __input, lazyllm_files = __input
    365         if isinstance(__input, str) and __input.startswith(LAZYLLM_QUERY_PREFIX):
    366             assert not lazyllm_files, 'Argument `files` is already provided by query'
    367             deinput = decode_query_with_filepaths(__input)
    368             __input, files = deinput['query'], deinput['files']
    369         else:
    370             files = _lazyllm_get_file_list(lazyllm_files) if lazyllm_files else []
    371
    372     query = __input
    373     __input = self._prompt.generate_prompt(query, llm_chat_history, tools)
    374     headers = {'Content-Type': 'application/json'}
    375     text_input_for_token_usage = __input
    376
    377     if isinstance(self, ServerModule):
    378         assert llm_chat_history is None and tools is None
    379         headers['Global-Parameters'] = encode_request(globals._pickle_data)
    380         headers['Session-ID'] = encode_request(globals._sid)
    381         data = encode_request((__input, kw))
    382     elif self.template_message:
```

```
● (lazyllm) (lazyllm) macro@CNBJWZHM examples % python llm.py
  { 'messages': [ { 'role': 'system', 'content': 'You are an AI assistant, developed by SenseTime.' }, { 'role': 'user', 'content': '你好' } ] }
○ (lazyllm) (lazyllm) macro@CNBJWZHM examples %
```



对比Alpaca和Chat

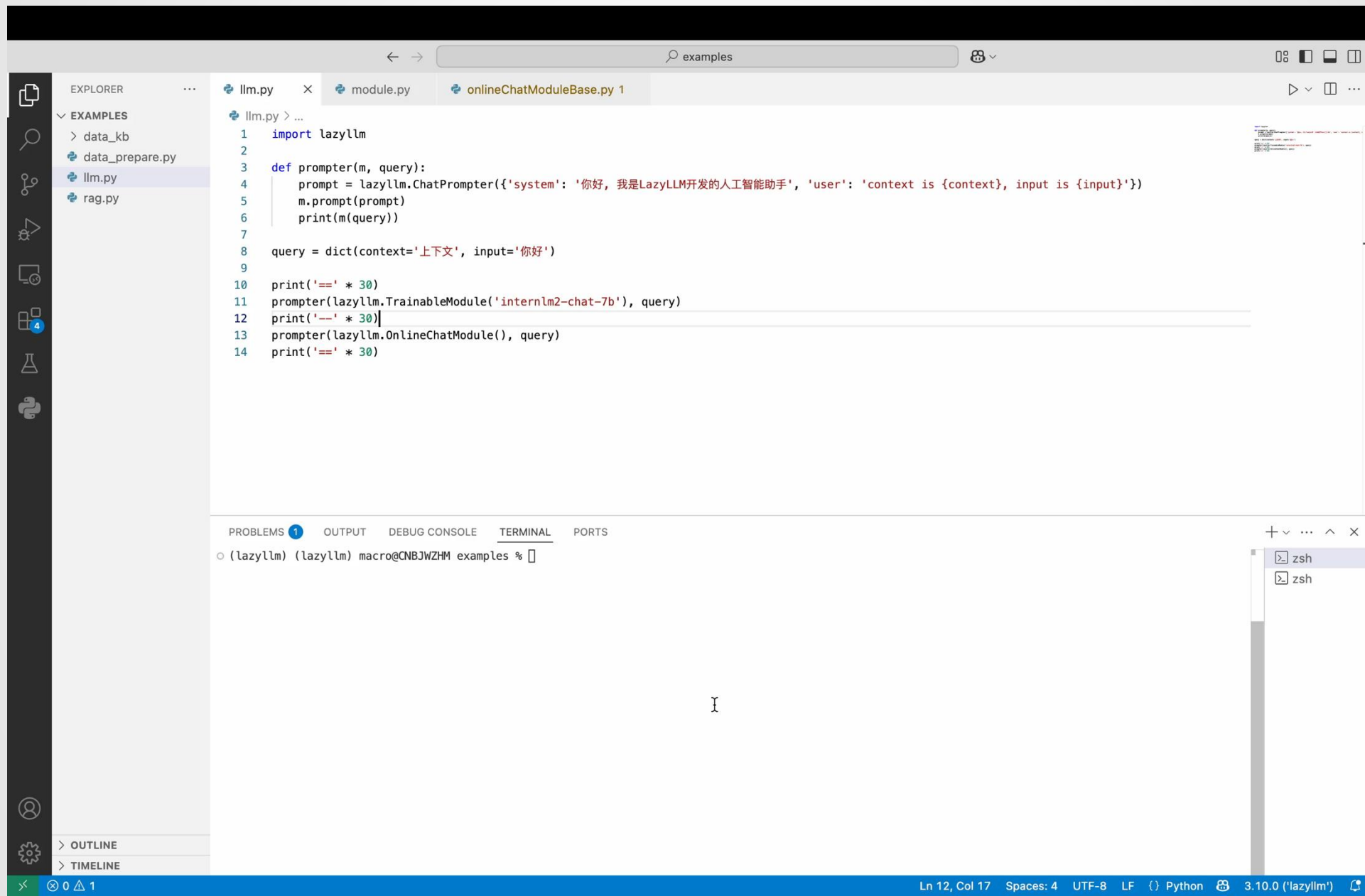
The screenshot shows a VS Code editor with the following components:

- EXPLORER:** A sidebar on the left showing a file tree with 'EXAMPLES' expanded, containing 'data_kb', 'data_prepare.py', 'llm.py' (selected), and 'rag.py'.
- EDITOR:** The main workspace showing the 'llm.py' file with the following code:

```
1 import lazylm
2
3 def prompter(p, query):
4     m = lazylm.TrainableModule('internlm2-chat-7b').prompt(p('你好，我是LazyLLM开发的人工智能助手，{input}'))
5     print(m(query))
6
7 query = '你好'
8
9 print('==' * 30)
10 prompter(lazylm.AlpacaPrompter, query)
11 print('--' * 30)
12 prompter(lazylm.ChatPrompter, query)
13 print('==' * 30)
```
- TERMINAL:** A terminal window at the bottom showing the command prompt 'o (lazylm) (lazylm) macro@CNBJWZHM examples %'.
- STATUS BAR:** The bottom status bar shows 'Ln 9, Col 17', 'Spaces: 4', 'UTF-8', 'LF', 'Python', and '3.10.0 ('lazylm')'.



对比Trainable和Online



The screenshot shows a VS Code editor with the following components:

- EXPLORER:** A sidebar on the left showing a file tree under 'EXAMPLES' with files: data_kb, data_prepare.py, llm.py (selected), and rag.py.
- Editor:** The main area displays the code in 'llm.py'. The code imports 'lazylm' and defines a 'prompter' function that uses 'lazylm.ChatPrompter'. It then creates a 'query' dictionary and calls 'prompter' with both 'TrainableModule' and 'OnlineChatModule'.
- TERMINAL:** A terminal window at the bottom shows the command prompt '(lazylm) (lazylm) macro@CNBJWZHM examples %'.
- STATUS BAR:** The bottom status bar indicates 'Ln 12, Col 17', 'Spaces: 4', 'UTF-8', 'LF', 'Python', and '3.10.0 ('lazylm')'.

```
1 import lazylm
2
3 def prompter(m, query):
4     prompt = lazylm.ChatPrompter({'system': '你好, 我是LazyLLM开发的人工智能助手', 'user': 'context is {context}, input is {input}'})
5     m.prompt(prompt)
6     print(m(query))
7
8 query = dict(context='上下文', input='你好')
9
10 print('=' * 30)
11 prompter(lazylm.TrainableModule('internlm2-chat-7b'), query)
12 print('-' * 30)
13 prompter(lazylm.OnlineChatModule(), query)
14 print('=' * 30)
```

o (lazylm) (lazylm) macro@CNBJWZHM examples %

Ln 12, Col 17 Spaces: 4 UTF-8 LF {} Python 3.10.0 ('lazylm')



目录



1. 上节回顾



2. LazyLLM介绍



3. 在LazyLLM中使用大模型



4. LazyLLM中的数据流



5. 借助数据流搭建你的RAG应用



LazyLLM数据流 – Why?

1. `from lazyllm import TrainableModule, pipeline`
2. `speech_rec = TrainableModule('SenseVoiceSmall')`
3. `translate = TrainableModule('internlm2-chat-7b')`
4. `drawing = TrainableModule('stable-diffusion-3-medium')`

`# 标准用法`

5. `speech_rec.start()`
6. `translate.start()`
7. `drawing.start()`
8. `def draw(query):`
9. `drawing(translate(speech_rec(query)))`

`# flow用法`

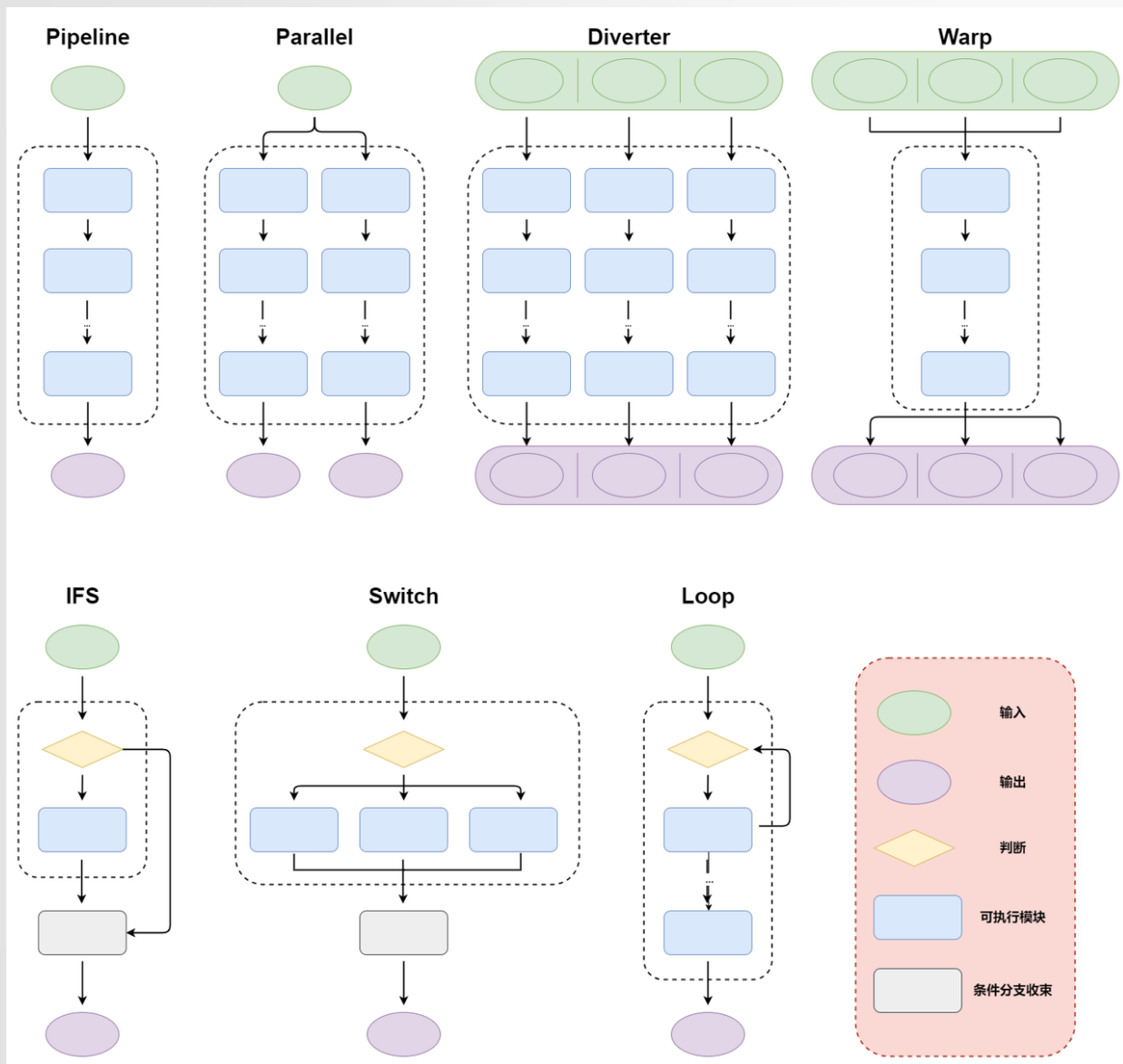
10. `draw = pipeline(speech_rec, translate, drawing).start()`

复杂应用一键部署

简化代码



LazyLLM数据流



- Pipeline: 顺序执行
- Parallel: 并发执行
- Diverter: 分流并发执行
- Warp: 数据并行 / 批处理
- IFS: 条件分支
- Switch: 多路选择
- Loop: 循环





Search



LazyAGI/LazyLLM
v0.4.1 1.6k 91

Home Cookbook Best Practice Advanced Topics Api Reference

Home

Getting Started

FAQ

Environment

Supported Models

Getting Started

Welcome to **LazyLLM**!

LazyLLM is an all-in-one development tool for building and optimizing multi-Agent applications. It provides a wealth of tools for all stages of application development (including application construction, data preparation, model deployment, model fine-tuning, evaluation, etc.), helping developers to build AI applications at a very low cost and continuously iterate and optimize the effects.

Table of contents

Environment Preparation

Manual Configuration

Using the Docker Image

Installing from Pip

Hello, world!

Going Further: Multi-turn Dialogue

Using the Web Interface

Using Command Line Interface

Environment Preparation

You can set up the **LazyLLM** development environment using any of the following methods:

Manual Configuration

LazyLLM is developed in Python, and we need to ensure that **Python**, **Pip**, and **Git** are already installed on our system.

First, prepare and activate a virtual environment named `lazyllm-venv`:

en stable

访达



这里我会直接录屏，到时候剪进去



目录

-  1. 上节回顾
-  2. LazyLLM介绍
-  3. 在LazyLLM中使用大模型
-  4. LazyLLM中的数据流
-  5. 借助数据流搭建你的RAG应用



用flow搭建RAG应用

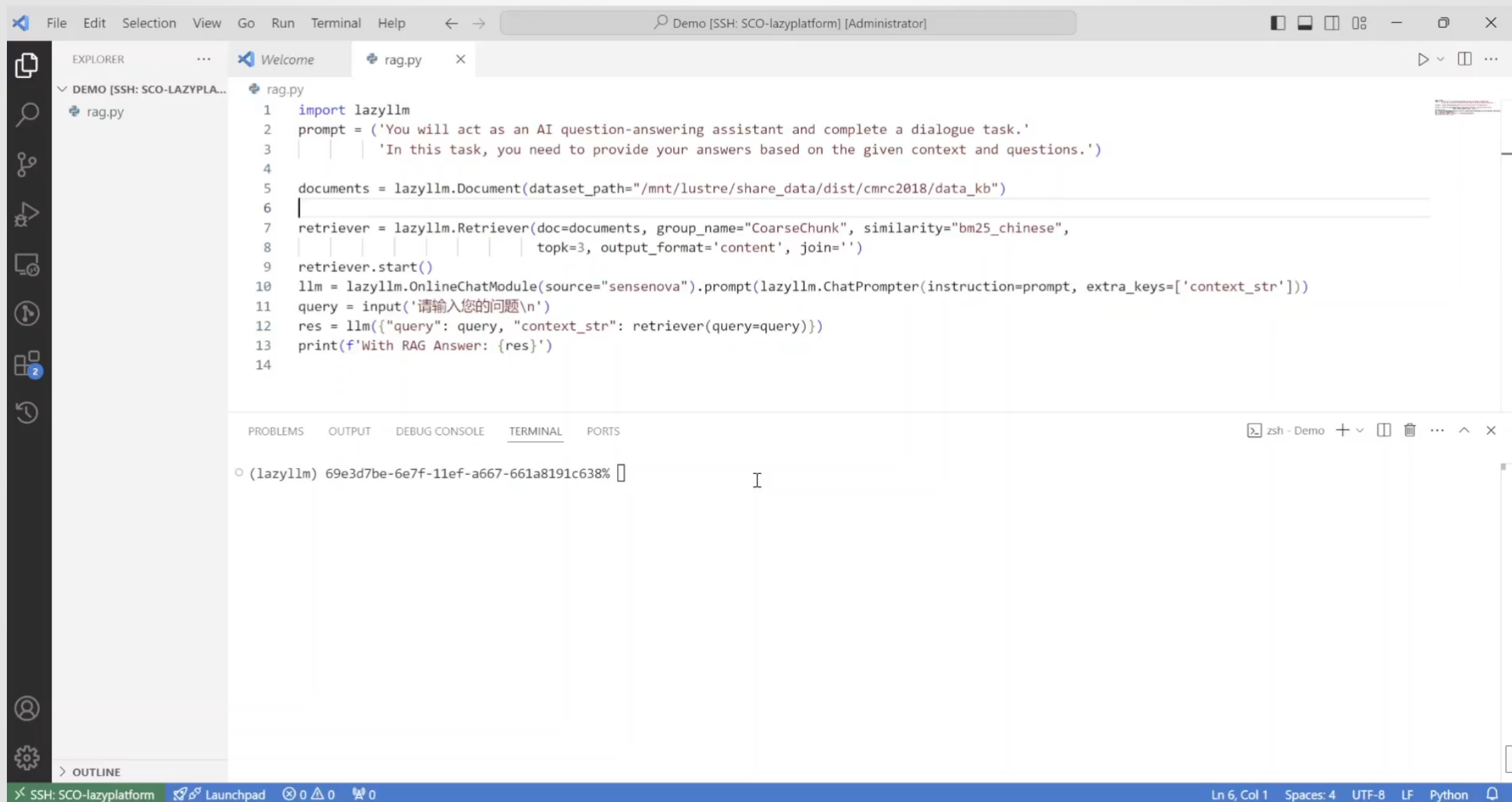
```
1. import lazyllm
2. prompt = ('You will act as an AI question-answering assistant and complete a dialogue task.'
            'In this task, you need to provide your answers based on the given context and questions.' )
3. documents = lazyllm.Document(dataset_path="/path/to/data_kb")
4. retriever = lazyllm.Retriever(doc=documents, group_name="CoarseChunk", similarity="bm25_chinese",
                                topk=3, output_format='content', join='')
5. retriever.start()
6. llm = lazyllm.OnlineChatModule(source="sensenova").prompt(lazyllm.ChatPrompter(instruction=prompt, extra_keys=['context_str']))
7. query = input('请输入您的问题\n')
8. res = llm({"query": query, "context_str": retriever(query=query)})
9. print(f'With RAG Answer: {res}')
```



```
1. import lazyllm
2. prompt = ('You will act as an AI question-answering assistant and complete a dialogue task.'
            'In this task, you need to provide your answers based on the given context and questions.')
3. documents = lazyllm.Document(dataset_path="/mnt/lustre/share_data/dist/cmrc2018/data_kb")
4. with lazyllm.pipeline() as ppl:
5.     ppl.retriever = lazyllm.Retriever(doc=documents, group_name="CoarseChunk", similarity="bm25_chinese",
                                        topk=3, output_format='content', join='')
6.     ppl.formatter = (lambda nodes, query: dict(context_str=nodes, query=query)) | bind(query=ppl.input)
7.     ppl.llm = lazyllm.OnlineChatModule("sensenova").prompt(lazyllm.ChatPrompter(instruction=prompt, extra_keys=['context_str']))
8. query = input('请输入您的问题\n')
9. res = ppl(query)
10. print(f'With RAG Answer: {res}')
```



用flow搭建RAG应用



```
File Edit Selection View Go Run Terminal Help Demo [SSH: SCO-lazyplatform] [Administrator]
EXPLORER DEMO [SSH: SCO-LAZYPLA... rag.py
rag.py
1 import lazyllm
2 prompt = ('You will act as an AI question-answering assistant and complete a dialogue task.'
3         'In this task, you need to provide your answers based on the given context and questions.')
4
5 documents = lazyllm.Document(dataset_path="/mnt/lustre/share_data/dist/cmrc2018/data_kb")
6
7 retriever = lazyllm.Retriever(doc=documents, group_name="CoarseChunk", similarity="bm25_chinese",
8                             topk=3, output_format='content', join='')
9 retriever.start()
10 llm = lazyllm.OnlineChatModule(source="sensenova").prompt(lazyllm.ChatPrompter(instruction=prompt, extra_keys=['context_str']))
11 query = input('请输入您的问题\n')
12 res = llm({"query": query, "context_str": retriever(query=query)})
13 print(f'With RAG Answer: {res}')
14

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS zsh - Demo
(lazyllm) 69e3d7be-6e7f-11ef-a667-661a8191c638%
Ln 6, Col 1 Spaces: 4 UTF-8 LF Python
```

Q&A

1. LazyLLM的数据流有什么作用，是否可以认为LazyLLM实际上是静态图
2. LazyLLM中的模型是否支持思维链模型的输出



感谢聆听
Thanks for Listening

